

CS.20002 문제해결기법

2주차 – HW1 풀이

TA 오태인

- KLMS의 문제 풀이 팁 모음이 업데이트되었습니다.
- 학습을 위한 OJ 사용법과 언어 레퍼런스 문서 등이 추가되었습니다.

HW1 풀이 – 계단 오르기 (easy)

- 수업 시간에 설명한 예제입니다.
- 처음에 1칸 또는 2칸을 오를 수 있으므로 다음 점화식을 얻습니다.

$$dp[n] = dp[n - 1] + dp[n - 2].$$

작은 n 부터 차례대로 계산하면 시간복잡도는 $\mathcal{O}(N)$ 입니다.

- 이 점화식을 메모이제이션 없이 재귀적으로 구현하면 7점을 받습니다.

- i 일까지 끝낸 과제들의 점수 합의 최댓값을 $dp[i]$ 라고 합시다.
- i 일에 끝나는 과제의 시작일 집합을 $\mathcal{S}$ 라 하면 다음이 성립합니다.

$$dp[i] = \max(\{dp[i-1]\} \cup \{dp[j-1] + S_j : j \in \mathcal{S}\}).$$

- 각 i 에 대해 $\mathcal{S}$ 를 구하기보다, $dp[T_i]$ 의 후보에 $dp[i-1] + S_i$ 를 넣는 것으로 생각합시다.

HW1 풀이 – 과제

- 아래와 같이 DP를 전이할 수 있습니다.

```
for (int i = 0; i <= n; i++) dp[i] = 0;
for (int i = 1; i <= n; i++) {
    dp[i] = max(dp[i], dp[i - 1]);
    dp[t[i]] = max(dp[t[i]], dp[i - 1] + s[i]);
}
```

- 정답은 $dp[n]$ 이며, 시간복잡도는 $\mathcal{O}(N)$ 입니다.
- 이처럼 DP는 닫힌 점화식으로 생각하지 않는 게 편할 때도 있습니다.

HW1 풀이 – 과제

- 점수를 `int` 형으로 저장하면 WA를 받습니다.
- 가능한 모든 조합을 직접 계산하면 6점을 받습니다.
- 정확히 i 일에 마지막 과제가 끝났을 때 얻을 수 있는 점수의 최댓값을 $dp[i]$ 라고 하면 다음 점화식을 얻습니다.

$$dp[i] = \max\{dp[j] + S_k : j < k, T_k = i\}.$$

그대로 계산하면 시간복잡도는 $\mathcal{O}(N^2)$ 이며, 29점을 받습니다.

- 수업 시간에 설명한 예제입니다.
- 각 열에 위쪽 선택, 아래쪽 선택, 선택하지 않음의 세 상태를 둡니다.
- i 번째 열까지 배정하고 i 번째 열의 상태가 j 일 때, 편안함의 최댓값을 $dp[i][j]$ 라고 하면 $\mathcal{O}(N)$ 에 문제를 풀 수 있습니다.
- 자세한 점화식은 지난 수업 자료를 참고해 주세요.

- i 이상의 데미지를 주기 위한 공격 횟수의 기댓값을 $dp[i]$ 라고 합시다.
- 주어진 확률 p/q 를 편의상 r 이라고 합시다.
- 다음 점화식을 얻습니다.

$$dp[i] = (1 - r)dp[i - 1] + rdp[i - x] + 1.$$

- 이론적으로는 $i \leq 0$ 일 때 $dp[i] = 0$ 으로 두면 됩니다. C++에서는 다음과 같이 구현합니다.

$$dp[i] = (1 - r)dp[i - 1] + rdp[\max(i - x, 0)] + 1.$$

- 모든 공격이 치명타라고 가정하고 $\lceil H/x \rceil$ 를 출력하면 3점을 받습니다.
- H 이상의 데미지를 줄 때까지 가능한 모든 케이스를 나열하고 직접 계산하면 7점을 받습니다.
- i 번 공격해 j 의 데미지를 입힐 확률을 $dp[i][j]$ 라고 한 뒤, 기댓값을 직접 계산하면 33점을 받습니다.

- 수업에서 설명한 LCS를 구현하면 되는 문제입니다.
- 모든 부분수열을 직접 비교하면 18점을 받습니다.

HW1 풀이 – 계단 오르기 (hard)

- 계단 오르기 (easy) 문제와 비슷하게 점화식을 세우면 됩니다.
- i 칸 올라가는 경우의 수를 $dp[i]$ 라고 하면 다음 점화식을 얻습니다.

$$dp[i] = \sum_{\substack{j \in S \\ j \leq i}} dp[i - j].$$

- 이를 계산하면 $\mathcal{O}(NM)$ 시간에 문제를 해결할 수 있습니다.
- 계단 오르기 (easy) 문제의 정답 코드를 제출하면 1점을 받습니다.
- 모든 경우를 나열하면 13점을 받습니다.

HW1 풀이 – 다이어트

- 수업 시간에 설명한 배낭 문제의 DP를 구현하면 됩니다.
- 맛의 값이 큰 것부터 채우면 6점을 받습니다.
- 가능한 모든 조합을 계산하면 11점을 받습니다.
- $dp[i][j]$ 를 i 번째 음식까지 고려해 열량 j 이하로 얻는 맛의 최댓값으로 두는 대신, 총 맛의 합을 j 이상으로 만드는 최소 열량으로 두고 계산하면 34점을 받습니다.

HW1 풀이 – 두바이 쏘든 쿠키

- 자기 자신도 연결된 것으로 보고, 1번과 2번 지점을 기준으로 집합을 나눕니다.
 - X : 1번에 연결되고 2번에는 연결되지 않은 제과점
 - Y : 2번에 연결되고 1번에는 연결되지 않은 제과점
 - Z : 1번과 2번에 모두 연결된 제과점
- 1번과 2번 지점이 각각 S 상자와 T 상자를 받으려면 X 에서 $S - a$ 개, Y 에서 $T - a$ 개, Z 에서 a 개의 제과점이 재료를 구해야 합니다.

HW1 풀이 – 두바이 쏘든 쿠키

- 집합 A 에서 i 개가 재료를 구할 확률을 $f_A[i]$ 라고 하면, 각 쿼리의 답은 다음과 같습니다.

$$\sum_{a=0}^{\min(S,T)} f_X[S-a]f_Y[T-a]f_Z[a].$$

- 따라서 각 쿼리를 $\mathcal{O}(N)$ 에 처리할 수 있습니다.
- 집합 A 안에서 제과점에 순서대로 번호를 붙이고, 첫 i 개 중 j 개가 재료를 구할 확률을 $\text{dp}_A[i][j]$ 라고 합시다.

$$\text{dp}_A[i][j] = (1 - P_i)\text{dp}_A[i-1][j] + P_i\text{dp}_A[i-1][j-1].$$

따라서 $f_A[i] = \text{dp}_A[|A|][i]$ 입니다.

- 전처리는 $\mathcal{O}(N^2)$ 이고, 전체 시간복잡도는 $\mathcal{O}(N^2 + NQ)$ 입니다.

HW1 풀이 – 두바이 쏘든 쿠키

- 모든 P_i 가 0 또는 1이면 받는 상자 수가 정해집니다. 이를 직접 계산하면 6점을 받습니다.
- 각 제과점의 성공 여부를 모두 나열해 계산하면 9점을 받습니다.
- 모든 P_i 가 P 로 같으면 다음 식으로 $f_A[i]$ 를 구할 수 있습니다.

$$f_A[i] = \binom{|A|}{i} P^i (1-P)^{|A|-i}.$$

이 분포로 쿼리를 $\mathcal{O}(N)$ 에 처리하면 14점을 받습니다.

- $Z = \emptyset$ 이면 답은 $f_X[S]f_Y[T]$ 이며, 쿼리당 $\mathcal{O}(1)$ 에 계산할 수 있습니다.
- 첫 i 개 제과점까지 고려해 1번이 j 상자, 2번이 k 상자를 받을 확률을 $\text{dp}[i][j][k]$ 로 두고 계산하면 32점을 받습니다.

HW1 풀이 – 과녁 색칠

- 빨간 페인트를 i 통 이하, 파란 페인트를 j 통 이하로 사용하여 만들 수 있는 크기 k 의 과녁의 수를 $dp[i][j][k]$ 라고 합시다.
- 크기의 상한을 K 라 하면 다음 점화식으로 $\mathcal{O}(abK)$ 에 계산합니다.

$$dp[i][j][k] = dp[i - k][j][k - 1] + dp[i][j - k][k - 1].$$

- k 를 $a + b$ 까지 순회하면 $\mathcal{O}(ab(a + b))$ 에 계산하며, 20점을 받습니다.
- $k(k + 1)/2 \leq a + b$ 임을 이용해 범위를 줄이면 시간복잡도는 $\mathcal{O}(ab\sqrt{a + b})$ 이며, 41점을 받습니다.

HW1 풀이 – 과녁 색칠

- 이번에는 빨간 페인트를 정확히 i 통, 파란 페인트를 정확히 j 통 사용하여 만들 수 있는 크기 k 의 과녁의 수를 $\text{dp}[i][j][k]$ 라고 합시다.
- $k(k+1)/2 = i+j$ 이므로 k 가 결정됩니다. 상태를 $\text{dp}[i][j]$ 로 줄이면 $\mathcal{O}(AB)$ 에 계산할 수 있습니다.
- 다음 누적합으로 빈 과녁을 제외한 합을 구합니다.

$$\text{pref}[i][j] = \sum_{r=0}^j \text{dp}[i][r].$$

$$\sum_{i=0}^a \sum_{j=0}^b \text{dp}[i][j] - 1 = \sum_{i=0}^a \text{pref}[i][b] - 1.$$

쿼리당 $\mathcal{O}(a)$ 에 계산하면 64점을 받습니다.

HW1 풀이 – 과녁 색칠

- 빨간 페인트 i 통을 쓴 크기 j 의 과녁 수를 $\text{dp}[i][j]$ 라고 합시다.
- 파란 페인트 사용량은 $j(j+1)/2 - i$ 통으로 정해지므로, 상태 수는 $\mathcal{O}(A\sqrt{A+B})$ 입니다.
- i 를 고정하고 j 방향의 누적합을 미리 구해 둡니다.

$$\text{pref}[i][j] = \sum_{r=1}^j \text{dp}[i][r].$$

- $j(j+1)/2 \leq x$ 를 만족하는 최대 j 를 $J[x]$ 로 미리 구해 둡니다.
- $j \leq J[b+i]$ 이므로 다음 합으로 쿼리를 $\mathcal{O}(a)$ 에 처리합니다.

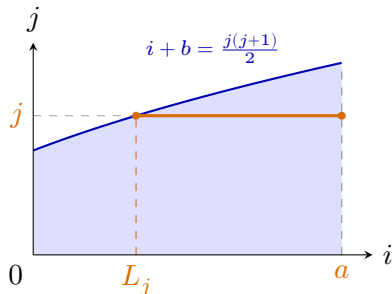
$$\sum_{i=0}^a \sum_{\substack{j \geq 1 \\ j(j+1)/2 - i \leq b}} \text{dp}[i][j] = \sum_{i=0}^a \text{pref}[i][J[b+i]].$$

HW1 풀이 – 과녁 색칠

- 전체 쿼리 수를 Q , 두 색의 최대 페인트 양을 각각 A, B 라고 합시다.
- 이 풀이의 시간복잡도는 $\mathcal{O}(A\sqrt{A+B} + QA)$ 이며, 90점을 받습니다.
- 쿼리의 답을 구할 때 j 를 고정하고 i 방향으로 누적합을 구해 봅시다.

$$\text{pref}[i][j] = \sum_{r=0}^i \text{dp}[r][j], \quad \text{pref}[-1][j] = 0.$$

HW1 풀이 – 과녁 색칠



크기 j 를 고정하면 $i = L_j$ 부터 a 까지 합하면 됩니다.

$$L_j = \max\left(0, \frac{j(j+1)}{2} - b\right).$$

- 가능한 최대 크기를 K 라고 하면, 쿼리의 답은 다음과 같습니다.

$$\sum_{j=1}^K (\text{pref}[a][j] - \text{pref}[L_j - 1][j]).$$

- 크기마다 $\mathcal{O}(1)$ 에 합을 구하므로, 쿼리당 $\mathcal{O}(\sqrt{a+b})$ 입니다.
- 전체 시간복잡도는 $\mathcal{O}((A+Q)\sqrt{A+B})$ 이며, 100점을 받습니다.

- 오는 길의 방향을 뒤집으면, 왕복 경로는 출발점과 도착점을 제외하고 같은 칸을 지나지 않는 두 최단 경로의 쌍과 대응합니다.
- 두 경로는 최단 경로이므로 같은 칸에서 만나려면 같은 횡수를 이동해야 합니다.
- i 번 이동한 뒤 첫 번째 경로가 (x_1, y_1) , 두 번째 경로가 (x_2, y_2) 에 위치하는 경우의 수를 $dp[i][x_1][y_1][x_2][y_2]$ 라고 합시다.
- 이 DP를 구현하면 11점을 받습니다.

- 두 경로의 위치는 다음 관계를 만족합니다.

$$x_1 + y_1 = x_2 + y_2 = i + 2.$$

- 따라서 상태를 $dp[x_1][y_1][x_2][y_2]$ 로 줄일 수 있습니다. 이 DP를 구현하면 34점을 받습니다.
- 사실 i, x_1, x_2 만 알아도 두 경로의 위치가 정해집니다.

$$y_1 = i + 2 - x_1, \quad y_2 = i + 2 - x_2.$$

- $dp[i][x_1][x_2]$ 로 상태를 줄이고, 이전 단계와 현재 단계의 배열만 저장하여 구현하면 100점을 받습니다.

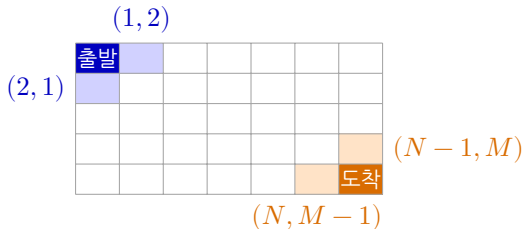
- DP 전이는 다음과 같습니다.

$$\text{dp}[i][x_1][y_1][x_2][y_2] = \sum \text{dp}[i-1][x_3][y_3][x_4][y_4].$$

- (x_3, y_3) 와 (x_4, y_4) 는 각각 (x_1, y_1) 과 (x_2, y_2) 의 위쪽 또는 왼쪽에 인접한 칸입니다.
- 장애물을 밟거나, 출발점과 도착점 외의 칸에서 두 경로가 만나는 상태의 DP 값은 0으로 둡니다.
- 격자 순회 구현은 KLMS의 문제 풀이 팁 모음을 참고해 주세요.
- $\min(N, M) = 2$ 이면 두 테두리를 따라가는 왕복 방법만 가능합니다. 장애물이 없으면 답은 2, 하나라도 있으면 0입니다.

HW1 풀이 – 산책

- 장애물이 없는 부분문제의 풀이는 다음과 같습니다.



- 다음 두 경로를 독립적으로 고릅니다.

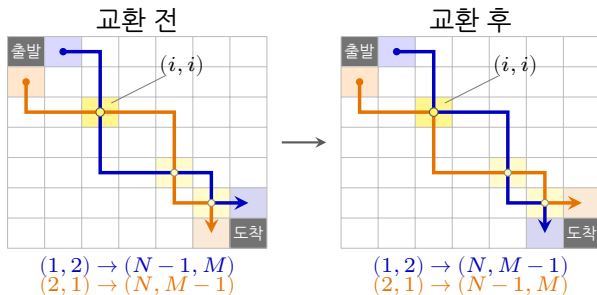
$$(1, 2) \rightarrow (N - 1, M), \quad (2, 1) \rightarrow (N, M - 1).$$

- 각 경로는 아래로 $N - 2$ 번, 오른쪽으로 $M - 2$ 번 이동합니다. 따라서 두 경로를 고르는 방법의 수는 다음과 같습니다.

$$\binom{N + M - 4}{N - 2}^2.$$

HW1 풀이 – 산책

- 이제 두 경로가 겹치는 경우를 빼줘야 합니다.
- 처음 만나는 칸 이후의 두 경로를 서로 맞바꿉니다.



- 그러면 서로 만나는 두 경로의 쌍은 도착점을 서로 바꿔 만든 두 경로의 쌍과 대응됩니다.

$$2 \left\{ \binom{N+M-4}{N-2}^2 - \binom{N+M-4}{N-1} \binom{N+M-4}{N-3} \right\}.$$

- 앞의 i 개 수를 j 개 구간으로 나눈 최대 스릴을 $dp[i][j]$ 라고 합시다.
- 마지막 구간의 시작점을 고르면 다음 점화식을 얻습니다.

$$dp[i][j] = \max_{0 \leq k < i} \{ dp[k][j-1] + \max_{k < r \leq i} A_r - \min_{k < r \leq i} A_r \}.$$

- 구간별 최댓값과 최솟값을 미리 구하면 $\mathcal{O}(N^3)$ 에 계산할 수 있습니다.
- 최댓값과 최솟값의 차라는 성질을 활용해 더 빠르게 풀어 봅시다.

- 문제를 더 일반적으로 바꿔 봅시다.
- 구간 $[l, r]$ 의 최댓값과 최솟값의 차이는 다음과 같습니다.

$$\max_{l \leq u \leq r} A_u - \min_{l \leq u \leq r} A_u = \max_{l \leq u, v \leq r} (A_u - A_v).$$

- 즉, 각 구간에서 더할 원소 A_u 와 뺄 원소 A_v 를 골라 전체 합을 최대화하는 문제로 바꿀 수 있습니다.
- i 번째 원소까지 처리했고, 현재 j 번째 구간에 있다고 합시다.
- 구간 j 에서 최댓값과 최솟값을 골랐는지를 각각 s, t 로 나타내고, 이때 얻을 수 있는 최댓값을 $dp[i][j][s][t]$ 라고 합시다.

HW1 풀이 – 롤러코스터

$$\text{dp}[i][j][0][0] = \max\{\text{dp}[i-1][j][0][0], \text{dp}[i-1][j-1][1][1]\}.$$

$$\text{dp}[i][j][1][0] = \max\{\text{dp}[i-1][j][1][0], \text{dp}[i][j][0][0] + A_i\}.$$

$$\text{dp}[i][j][0][1] = \max\{\text{dp}[i-1][j][0][1], \text{dp}[i][j][0][0] - A_i\}.$$

$$\text{dp}[i][j][1][1] = \max \left\{ \begin{array}{l} \text{dp}[i-1][j][1][1], \\ \text{dp}[i][j][1][0] - A_i, \\ \text{dp}[i][j][0][1] + A_i \end{array} \right\}.$$

- 각 상태를 $\mathcal{O}(1)$ 에 계산하므로 전체 시간복잡도는 $\mathcal{O}(N^2)$ 입니다.
- 정확히 K 개 구간으로 나눈 답은 $\text{dp}[N][K][1][1]$ 입니다. 이렇게 구현하면 15점을 받습니다.

휴강 공지

- 10/9는 한글날로 휴강합니다.
- 휴강인데 풀 문제가 없어서 심심하신 분들은 DOJ의 ‘**DOJ Contest 7**’, ‘**DOJ Beginner Contest 11**’ 등 온라인 대회에 참가해 보는 것을 추천드립니다.
- 사유: 제가 일부 출제했습니다.

